

Advantages of OOP

- Code reusability
- Data security
- Easy maintenance
- Modularity
- Better scalability
- Faster software development

Conclusion

The four OOP principles—**Encapsulation, Abstraction, Inheritance, and Polymorphism**—are the foundation of object-oriented programming in Python. They help create secure, reusable, and maintainable applications.

This answer is suitable for a 10–15 mark university exam.



 Messages beyond this point are only visible to you

Define class and object? How to create the class and object? How to access the members of class using the object reference?

Define Class and Object. How to Create the Class and Object? How to Access the Members of a Class Using the Object Reference? (Python) (10–15 Marks)

Class

A **class** is a blueprint or template used to create objects. It defines the **data (attributes)** and **functions (methods)** that the objects will have.

Syntax:

```
class ClassName:
    # Attributes and methods
    pass
```

Object

An **object** is an instance of a class. It occupies memory and can access the attributes and methods of the class.

Syntax:

Python

Run

Creating a Class and Object in Python

```
class Student:
    def __init__(self, name, roll):
        self.name = name
        self.roll = roll

    def display(self):
        print("Name:", self.name)
        print("Roll No:", self.roll)

# Creating an object
s1 = Student("Abdul", 101)

# Calling method
s1.display()
```

Output:

```
Name: Abdul
Roll No: 101
```

Accessing Members of a Class Using the Object Reference

In Python, class members are accessed using the dot (.) operator.

Example:

```
class Student:
    def __init__(self):
        self.name = "Rahul"
        self.age = 20

    def show(self):
        print(self.name)
        print(self.age)

obj = Student()

# Accessing data members
print(obj.name)
print(obj.age)

# Accessing method
obj.show()
```

Output:

```
Rahul
20
Rahul
20
```

Explanation

- `class Student:` → Defines a class.
- `__init__()` → Constructor used to initialize object data.
- `self` → Refers to the current object.
- `obj = Student()` → Creates an object.
- `obj.name` → Accesses the data member.
- `obj.show()` → Calls the member function.

Advantages of Classes and Objects

- Code reusability.
- Easy maintenance.
- Better organization of programs.
- Supports Object-Oriented Programming (OOP).
- Improves readability and scalability.

Conclusion

A **class** is a blueprint for creating objects, while an **object** is an instance of a class. In Python, objects are created using the class name, and class members (attributes and methods) are accessed using the **dot (.) operator**. This is the standard way to implement OOP concepts in Python.

This answer is suitable for a 10–15 marks university exam.



Do you like this personality?

